



API SECURITY

RISK ANALYSIS

RISK = Likelihood × Impact

Prepared by
TAMIL SELVAN N

Why API Security Matters

APIs are the connective tissue of modern software — powering mobile apps, microservices, partner integrations, and IoT. That same connectivity makes them one of the most exposed and frequently attacked layers of the stack.

83%

of web traffic today is API calls, not
traditional browser traffic

#1

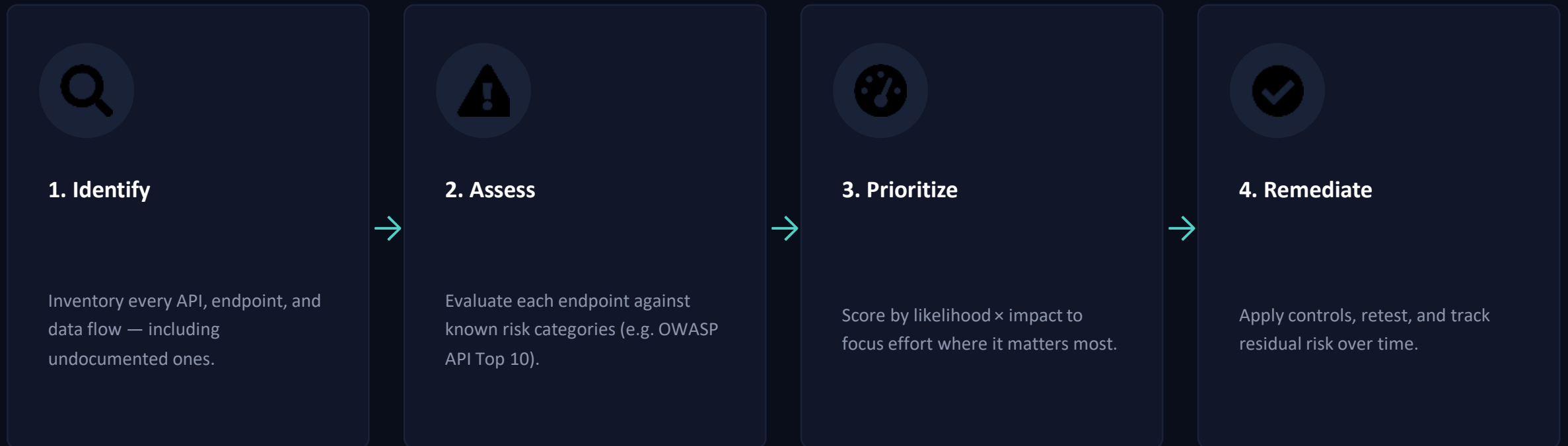
attack vector projected for enterprise app
breaches

1000s

of undocumented “shadow” API endpoints
in the average org

The Risk Analysis Framework

A structured approach turns a long list of possible flaws into a prioritized action plan.



OWASP API Security Top 10

The most widely referenced baseline for classifying and prioritizing API risks.

- | | | | |
|----|--|----|---|
| 01 | Broken Object Level Authorization | 06 | Unrestricted Access to Sensitive Business Flows |
| 02 | Broken Authentication | 07 | Server-Side Request Forgery (SSRF) |
| 03 | Broken Object Property Level Authorization | 08 | Security Misconfiguration |
| 04 | Unrestricted Resource Consumption | 09 | Improper Inventory Management |
| 05 | Broken Function Level Authorization | 10 | Unsafe Consumption of APIs |

The next several slides cover the highest-impact risks in detail.

Broken Object Level Authorization

CRITICAL

The most common and highest-impact API risk. Occurs when an API lets users access objects (records, files, accounts) belonging to others by simply changing an ID in the request.

HOW IT HAPPENS

An endpoint like `/api/orders/{id}` trusts the ID in the URL without verifying the requesting user actually owns that record.

POTENTIAL IMPACT

Unauthorized data disclosure, account takeover, or modification of another user's records at scale.

**Vulnerable Request**

```
GET /api/orders/1001
Authorization: Bearer <userA_token>

# Attacker changes ID:
GET /api/orders/1002
# Returns userB's order
```

Broken Authentication

02

CRITICAL

Weaknesses in how an API verifies identity — weak tokens, missing rate limits on login, or flawed password/credential handling that let attackers impersonate legitimate users.

HOW IT HAPPENS

APIs often reimplement authentication independently of the main web app, missing protections that exist elsewhere — like lockouts or MFA.

POTENTIAL IMPACT

Full account compromise, credential stuffing at scale, and impersonation of legitimate API clients.



Common Weaknesses

- No rate limiting on /login
- Weak or predictable API keys
- JWTs with no expiry
- Secrets exposed in client code

Broken Object Property Level Auth

HIGH

APIs often return full data objects and rely on the client to filter what's displayed. Attackers simply skip the UI and read the raw response, exposing fields never meant to be shared.

HOW IT HAPPENS

A `/users/{id}` endpoint returns an entire user record — including internal fields like password hashes, SSNs, or admin flags — relying on the frontend to hide them.

POTENTIAL IMPACT

Leakage of sensitive personal, financial, or internal data — often unnoticed until a breach disclosure.



Example Response

```
{
  "id": 42,
  "email": "user@co.com",
  "is_admin": false,
  "password_hash": "...",
  "ssn": "xxx-xx-xxxx"
}
```

Unrestricted Resource Consumption

HIGH

APIs that don't limit how many requests, how much data, or how many resources a client can consume are vulnerable to abuse — intentional or not.

HOW IT HAPPENS

Missing limits on request rate, payload size, pagination, or execution timeouts let a single client exhaust shared infrastructure.

POTENTIAL IMPACT

Denial of service, inflated cloud costs, and degraded performance for all legitimate users.



Mitigating Controls

- Per-client rate limiting
- Max payload size
- Pagination limits on list endpoints
- Timeouts on long-running calls

Broken Function Level Authorization

CRITICAL

Occurs when an API fails to properly restrict which functions a given user role can call — letting a regular user invoke admin-only operations directly.

HOW IT HAPPENS

Admin endpoints are often just “hidden,” not actually access-controlled, assuming users won't guess the URL or method.

POTENTIAL IMPACT

Privilege escalation — a standard user gains admin capabilities like deleting accounts or changing permissions.



Vulnerable Pattern

```
POST /api/users/42/role
{ "role": "admin" }

# No server-side check that
# caller is themselves an admin
```

Server-Side Request Forgery (SSRF)

HIGH

Happens when an API fetches a remote resource based on a user-supplied URL without validation, letting attackers redirect the server to internal systems.

HOW IT HAPPENS

A feature like “import from URL” or webhook validation passes attacker-controlled input straight into an outbound request.

POTENTIAL IMPACT

Access to internal services, cloud metadata endpoints, and lateral movement inside the network perimeter.



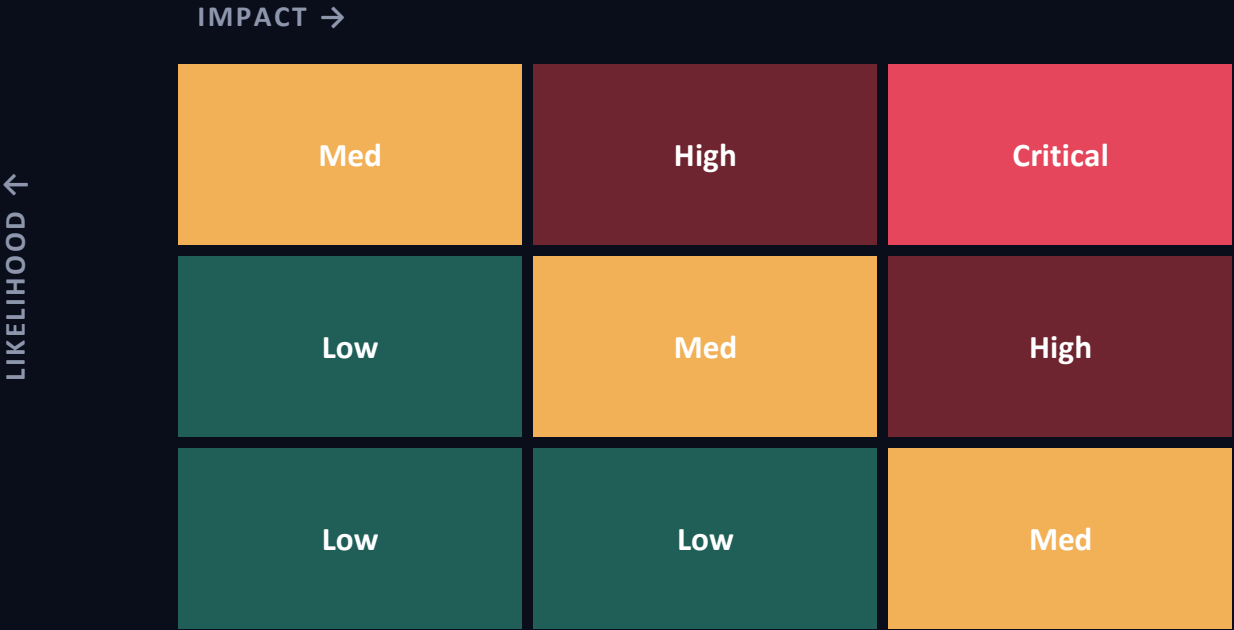
Malicious Input

```
POST /api/fetch-avatar
{ "url": "http://169.254.169.254/
  latest/meta-data/" }

# Server fetches internal
# cloud metadata for attacker
```

Likelihood × Impact Risk Matrix

Plot each identified risk to focus remediation on what matters most first.



Example Placements

- BOLA Critical
- Broken Auth Critical
- Resource Abuse High
- SSRF High
- Misconfig (low use) Medium

Common Attack Vectors



Parameter Tampering

Modifying IDs, prices, or roles in requests the client wasn't meant to control.



Fuzzing Undocumented Endpoints

Discovering shadow or deprecated API versions still exposed and unpatched.



Credential Stuffing

Automated login attempts using leaked credential lists against weakly protected auth endpoints.



Man-in-the-Middle on Mobile APIs

Intercepting traffic from apps that skip certificate pinning or use weak TLS config.

Mitigation Best Practices



Enforce object & function-level authorization

Verify ownership and role server-side on every request, never trust client-supplied IDs.



Apply strong authentication & token hygiene

Short-lived tokens, MFA where possible, and strict secret management.



Rate limit and validate input

Throttle by client, cap payload sizes, and validate every field against a strict schema.



Maintain a full API inventory

Track every version and endpoint, retire unused ones, and monitor for shadow APIs.



Log, monitor, and test continuously

Centralize API logs, alert on anomalies, and run regular security testing.



Shift Left

Bake API security checks into CI/CD — schema validation, static analysis, and automated authorization tests — so risks are caught before deployment, not after an incident.

API Risk Analysis Checklist

☐

Is every object-returning endpoint checked for caller ownership?

☐

Are admin/privileged functions gated by server-side role checks?

☐

Do responses return only the fields the client actually needs?

☐

Are rate limits and payload caps enforced per client?

☐

Is every API version and endpoint documented and inventoried?

☐

Are outbound requests from the API validated against an allowlist?

☐

Is authentication tested against brute-force and credential stuffing?

☐

Are API logs centralized and monitored for anomalies?

KEY TAKEAWAYS

Treat API Risk as a Continuous Process



APIs are the new perimeter

Every endpoint is a potential entry point — map them all, including shadow APIs.



Prioritize by risk, not alphabet

Use likelihood × impact to focus effort on BOLA, auth, and privilege issues first.



Authorization is non-negotiable

Object- and function-level checks belong on the server, on every single request.



Security is ongoing, not one-time

Continuous testing, monitoring, and inventory keep pace with how fast APIs change.

Next step: run an API inventory audit and map each endpoint against the OWASP API Top 10.